A COMPLETE VISUAL GUIDE

AdaBoost

Adaptive Boosting — from first principles to production tuning

Weak Learners | Decision Stumps | Boosting Math | Hyperparameters | Bagging vs Boosting

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \, h_t(x) \right)$$

Basic → Intermediate → Advanced · Worked numerical example included · Dark Mode Edition

CONTENTS

What's inside this guide

01	Foundations — Weak Learners, Decision Stumps, Labels	Basic
02	The Core Idea — Stage-wise Additive Modeling	Basic
03	Why Alpha? The "Which Friend Do You Trust" Intuition	Basic
04	Full Worked Example — AdaBoost by Hand (N = 5)	Intermediate
05	Upsampling — Turning Mistakes into Emphasis	Intermediate
06	Combining Models — The Final Prediction	Intermediate
07	Geometric Intuition — From Straight Lines to Curves	Intermediate
08	Hyperparameters & Tuning	Advanced
09	GridSearchCV in Practice	Advanced
10	Bagging vs. Boosting — The Real Difference	Advanced / Pro
11	Cheat Sheet & Where to Go Next	Reference

HOW TO READ THIS GUIDE

Every part builds on the one before it. Sections 1–3 give you the vocabulary. Section 4 is the heart of the guide — a complete AdaBoost run computed by hand on a tiny dataset. Sections 8–10 are "pro level": tuning knobs, grid search, and how AdaBoost compares to Random Forest style bagging.

PART 01 · BASIC

Prerequisites and Foundational Concepts

AdaBoost (**Ada**ptive **Boost**ing) is a foundational machine learning algorithm introduced roughly 25 years ago. It originally dominated early computer-vision tasks like face detection. Deep learning has since taken over that domain, but AdaBoost remains highly effective on tabular data — and mastering it is the essential gateway to understanding modern algorithms like Gradient Boosting and XGBoost.

WHERE IT FITS

Random Forest (a "bagging" method) may often edge AdaBoost out in raw accuracy on a given dataset, but AdaBoost teaches you the mechanics — sequential correction, weighted voting — that power almost every modern boosting library.

Before touching the algorithm itself, you need three building blocks firmly in place.

1.1 Weak Learners

DEFINITION

A **weak learner** is a model whose accuracy is only slightly better than random guessing. If you flip a coin, you're right 50% of the time — a weak learner is anything that clears that bar by just a little.

AdaBoost's entire power comes from combining *hundreds* of these individually weak, barely-better-than-chance models into a single, highly accurate ensemble.

1.2 Decision Stumps

AdaBoost can technically use any weak learner, but in practice **99% of implementations use decision stumps** because their behaviour is instantly visual and interpretable. A decision stump is simply a decision tree with **max depth = 1** — it is allowed to make exactly one split.

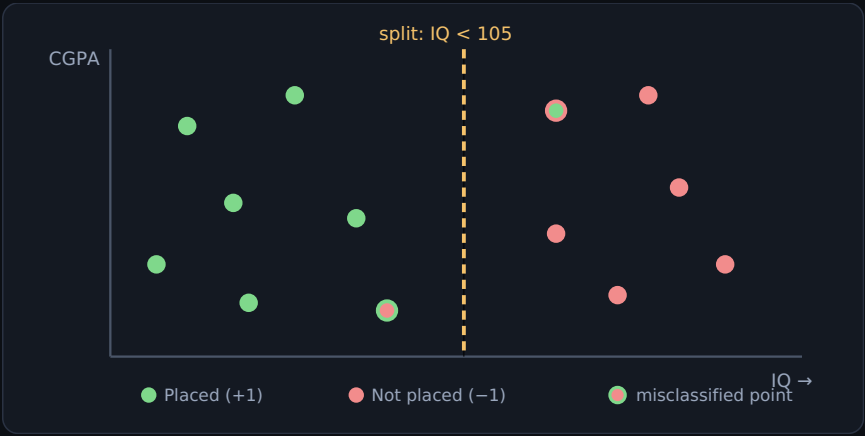


Fig 1.1 — A decision stump on a CGPA (Y) vs. IQ (X) scatter plot. It draws exactly ONE straight line, parallel to an axis, chosen to give the maximum drop in entropy (maximum information gain). Two points end up on the wrong side.

VISUAL CONCEPT

Picture a 2-D scatter plot of students: CGPA on the Y-axis, IQ on the X-axis. A decision stump draws exactly one straight line across the graph, parallel to either the X- or Y-axis, choosing whichever split maximises information gain (equivalently, maximises the reduction in entropy / Gini impurity).

1.3 Binary Labels: +1 and −1

Unlike ordinary binary classification, which is happy to use 0 and 1, AdaBoost **strictly requires** class labels of **+1** for the positive outcome (e.g. "placed") and **−1** for the negative outcome (e.g. "not placed"). This choice isn't cosmetic — it's what lets the final prediction be computed with simple addition and a *sign* function, as you'll see in Part 6.

PART 02 · BASIC

The Core Algorithm: Stage-Wise Additive Modeling

AdaBoost is formally a **stage-wise additive method**. Break that phrase into its two halves:

ADDITIVE = the strong model is built by adding weak learners together

STAGE-WISE = learners are added sequentially, each learning from the last one's mistakes

2.1 How one stage flows into the next

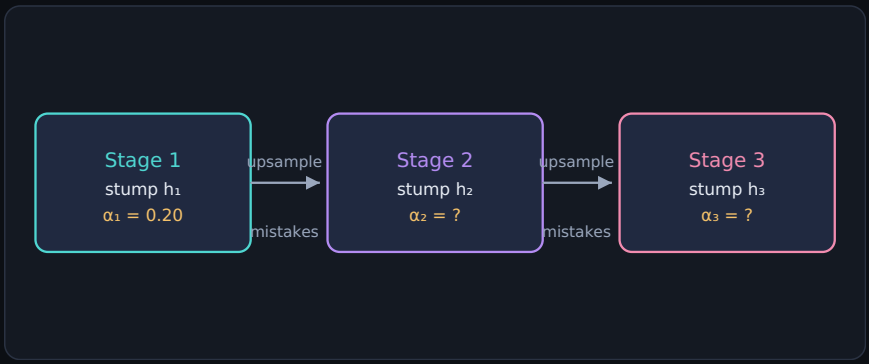


Fig 2.1 — Each stage trains a fresh stump, earns a weight α, then boosts the rows it got wrong before handing off to the next stage.

2.2 Stage 1

The first decision stump looks at the data (e.g. the CGPA / IQ plot) and makes its single optimal split. Because it is, by definition, a weak learner, it *will* misclassify some points — for example, three **+1** points might mistakenly fall into the negative region.

2.3 Calculating the model's weight, α

Unlike bagging methods such as Random Forest — where every base model gets one equal, democratic vote — boosting is a **meritocracy**. Each stump earns a weight α, based on its error rate: a highly accurate stump gets a large α, a poor one gets a small α. The full derivation is in Part 3.

2.4 Upsampling the mistakes

The points Stage 1 got wrong are passed into an **upsampling** mechanism: their "importance" (weight) in the dataset is deliberately increased, so the next stump is heavily penalised if it ignores them again.

2.5 Repeat

The updated, upsampled dataset feeds Stage 2. Its stump creates a brand-new split, engineered specifically to fix the three points Stage 1 missed — but in doing so, it will inevitably misclassify some *new* points. Compute α_2, upsample the new mistakes, and hand off to Stage 3. This loop continues for as many stages (stumps) as you choose.

PART 03 · BASIC

Why Alpha? The "Which Friend Do You Trust" Intuition

Before writing down a formula, build the intuition the way you'd reason about it yourself. Imagine three friends who each answer a yes/no question for you:

Friend	Error rate	How reliable?
A	0% (never wrong)	**Extremely reliable**
B	100% (always wrong)	**Also extremely reliable — just believe the opposite!**
C	50% (coin flip)	**Useless — carries zero information**

THE TWIST

Most people's gut instinct says Friend C (50% error, i.e. sometimes right) is "safer" than Friend B (always wrong). But Friend B is a goldmine — you simply flip his answer, and he becomes exactly as reliable as Friend A. Friend C is the truly dangerous one: he carries no usable signal at all.

This tells you exactly what shape the function mapping *error* → *α* needs to have:

- error ≈ 0 → α should be a **large positive** number (strong, trusted vote)
- error = 0.5 → α should be **0** (no vote at all)
- error ≈ 1 → α should be a **large negative** number (trust the opposite)

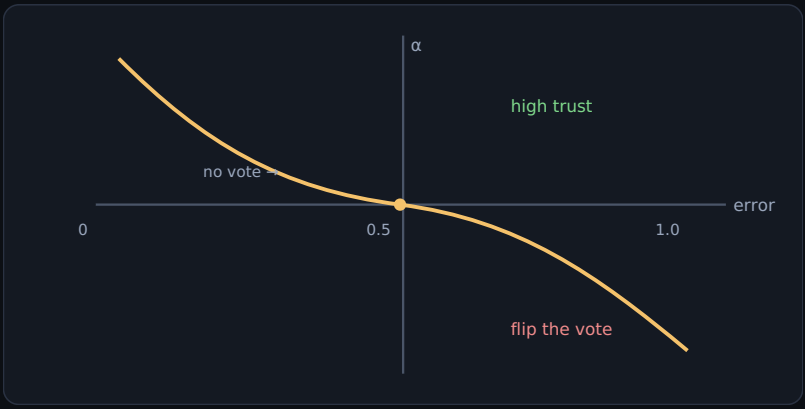


Fig 3.1 — The required error → α curve: steeply positive near error = 0, crosses zero at error = 0.5, and swings steeply negative as error → 1.

The mathematical function that produces exactly this curve is:

ALPHA (MODEL WEIGHT) FORMULA

$$\alpha = \frac{1}{2} \ln \frac{(1 - \text{error})}{\text{error}}$$

SANITY CHECK

Plug in error = 0.5: (1 − 0.5)/0.5 = 1, and ln(1) = 0, so α = 0. Exactly what we wanted — a coin-flip stump gets zero say in the final vote.

3.1 How "error" itself is calculated

This is *not* a plain accuracy count. The error used inside the α formula is the **sum of the weights** of every row the stump misclassified:

WEIGHTED ERROR OF A STUMP

$$\text{error} = \Sigma \text{ (weight of each misclassified row)}$$

Because every row starts with an equal weight (more on this in Part 4), and misclassified rows get boosted after each stage, later stumps are graded on a dataset that increasingly emphasises the points earlier stumps got wrong.

PART 04 · INTERMEDIATE

Full Worked Example — AdaBoost by Hand

Theory only clicks once you've pushed the numbers through yourself. Here is a complete, tiny, from-scratch pass of AdaBoost's first stage, on a dummy dataset with two input columns (X_1, X_2) and a label column $Y \in \{+1, -1\}$, with **N = 5** rows.

Step 1 · Initialise row weights

Every row starts with an **equal** weight of 1/N.

$$\text{weight}_i = 1 / N = 1 / 5 = 0.2 \quad \text{for every row } i$$

Row	X_1	X_2	Y (actual)	Weight
1	–	–	**+1**	0.20
2	–	–	**−1**	0.20
3	–	–	**+1**	0.20
4	–	–	**−1**	0.20
5	–	–	**+1**	0.20

(1/N added five times always sums to exactly 1 — this is why weights are initialised this way.)

Step 2 · Train the first decision stump (M1)

Train a depth-1 decision tree. It will cut on either X_1 or X_2 (e.g. "$X_1 < 5$" or "$X_2 < 7$"). Many possible stumps exist — pick whichever gives the **maximum decrease in entropy / maximum information gain**. Call the winner **Model M1**, and run every row through it to get its predictions.

Row	Y (actual)	M1 prediction	Correct?	Weight
1	**+1**	**+1**	✓	0.20
2	**−1**	**−1**	✓	0.20
3	**+1**	**−1**	✗	0.20
4	**−1**	**+1**	✗	0.20
5	**+1**	**+1**	✓	0.20

Step 3 · Compute the weighted error

Rows 3 and 4 are misclassified. Sum *their* weights:

$$\text{error} = \text{weight}_3 + \text{weight}_4 = 0.2 + 0.2 = \mathbf{0.4}$$

Step 4 · Compute α_1

$$\alpha_1 = \frac{1}{2} \ln \frac{1 - 0.4}{0.4} = \frac{1}{2} \ln \frac{0.6}{0.4} \approx \mathbf{0.20}$$

Step 5 · Update every row's weight

Two different update rules apply, depending on whether the row was classified correctly:

CORRECTLY CLASSIFIED ROWS

$$\text{new weight} = \text{old weight} \times e^{-\alpha}$$

$= 0.2 \times e^{-0.20} \approx \mathbf{0.16}$

MISCLASSIFIED ROWS

$$\text{new weight} = \text{old weight} \times e^{+\alpha}$$

$= 0.2 \times e^{+0.20} \approx \mathbf{0.24}$

Notice the pattern: correct rows shrink, wrong rows grow — exactly the "pay more attention to what I missed" signal boosting is built on.

Step 6 · Normalise so weights sum to 1

Add the five new (unnormalised) weights: 0.16 + 0.16 + 0.24 + 0.24 + 0.16 = **0.96**, not 1. Divide every weight by 0.96 to renormalise.

Row	Correct?	Raw new weight	Normalised (÷ 0.96)
1	✓	0.16	0.167
2	✓	0.16	0.167
3	✗	0.24	**0.25**
4	✗	0.24	**0.25**
5	✓	0.16	0.167

CHECKPOINT

After Stage 1 you have three deliverables: the trained stump M1, its vote weight $\alpha_1 \approx 0.20$, and a freshly reweighted dataset where rows 3 and 4 now matter roughly 50% more than rows 1, 2 and 5.

PART 05 · INTERMEDIATE

Upsampling — Turning Weights into a New Dataset

Row weights alone don't retrain a stump — most implementations convert the weights into a physically resampled dataset, a step called **upsampling**. The intuition: rows with higher weight should appear *more often* in the data Stage 2 actually trains on.

5.1 Build cumulative ranges

Lay the normalised weights end-to-end as cumulative ranges along the 0–1 interval:

Row	Weight	Cumulative range
1	0.167	0.00 – 0.167
2	0.167	0.167 – 0.334
3	**0.25**	0.334 – 0.584
4	**0.25**	0.584 – 0.834
5	0.167	0.834 – 1.00

5.2 Roll N random numbers between 0 and 1

Generate N = 5 random numbers, and for each one, find which row's range it falls into. Wider ranges (higher-weight, misclassified rows) are simply more likely to "catch" a random number.

Random draw	Falls in range of	Row selected
0.10	0.00–0.167	Row 1
0.43	0.334–0.584	**Row 3**
0.60	0.584–0.834	**Row 4**
0.52	0.334–0.584	**Row 3**
0.90	0.834–1.00	Row 5

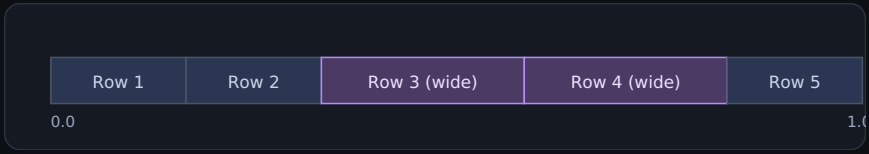


Fig 5.1 — Misclassified rows 3 and 4 occupy wider slices of the 0–1 range, so random draws land on them more often.

The new training set for Stage 2 is therefore: **Row 1, Row 3, Row 4, Row 3, Row 5** — row 3 now appears twice, row 4 once, and row 2 (a correctly classified row) has dropped out entirely by chance. The previously-wrong points now dominate the data Stage 2's stump is trained and graded on.

WHAT HAPPENS NEXT

Train a brand-new stump on this resampled data → get a new split (the data changed, so the optimal cut changes too) → compute its weighted error → compute α_2 → upsample its mistakes → hand off to Stage 3. This loop repeats for as many stumps as you configure (the `n_estimators` hyperparameter — see Part 8). At the end you have a chain: α_1, α_2, α_3, …, α_T.

PART 06 · INTERMEDIATE

Combining Models — The Final Prediction

Once every stage is complete, you're left with a collection of base models $h_t(x)$ and their voting weights α_t. To predict on new data, AdaBoost sums the *weighted* outputs of every base model, then takes the mathematical sign of the result.

ADABOOST'S HYPOTHESIS FUNCTION

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \, h_t(x) \right)$$

6.1 Worked example — three base models

A new student has CGPA = 7.5 and IQ = 10. Three trained stumps vote:

Model	Prediction $h_t(x)$	Weight α_t	Contribution
Model 1	**−1 (No placement)**	2	2 × (−1) = −2
Model 2	**+1 (Placement)**	10	10 × (+1) = +10
Model 3	**−1 (No placement)**	1	1 × (−1) = −1

$$2(-1) + 10(1) + 1(-1) \;=\; -2 + 10 - 1 \;=\; 7$$

FINAL ANSWER

sign(7) = **+1**. Even though two of the three models voted "No placement," Model 2's huge weight (α = 10, meaning it was far more accurate than the others) overrides them. The ensemble predicts the student **will** secure a placement — this is the meritocracy in action.

This is precisely why AdaBoost is *not* majority-vote democracy: a single highly-trusted stump can outvote several weaker ones. Contrast this with Random Forest, covered in Part 10, where every tree's vote counts equally regardless of individual accuracy.

PART 07 · INTERMEDIATE

Geometric Intuition — From Straight Lines to Curves

AdaBoost's real "aha" moment is visual. A single decision stump can only draw a rigid, straight line — horizontal or vertical. But stack dozens of these weighted lines on top of each other, and their overlapping boundaries merge into a highly complex, custom-fitted decision boundary that curves and wraps around the data clusters.

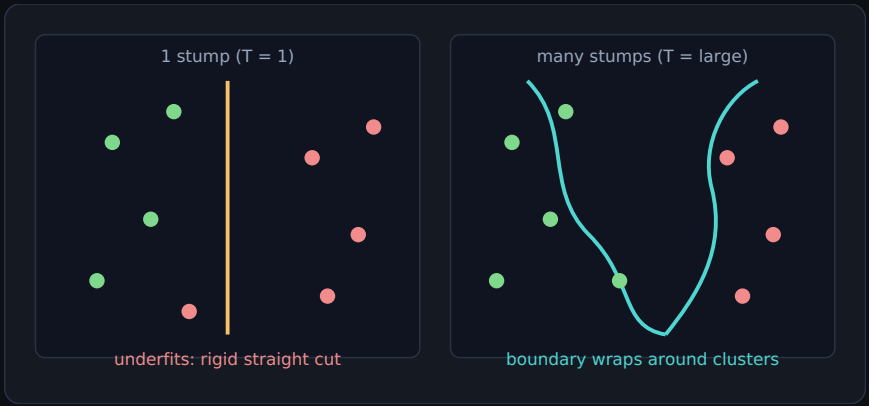


Fig 7.1 — One stump (left) can only cut the space in half. Many weighted stumps (right) combine into a boundary that curves naturally around each class.

This is the same geometric principle you'll revisit in Part 8 when tuning `n_estimators`: too few stumps and the boundary underfits (too rigid); too many and it starts drawing tiny islands around single stray points (overfitting).

BIG PICTURE SO FAR

Stage-wise boosting is a loop of: *fit → measure mistakes → weight the model → re-emphasise the mistakes → repeat.* The next step is peeling back the abstraction and controlling that loop directly — the hyperparameters in Part 8.

PART 08 · ADVANCED

Hyperparameters & Practical Tuning

Where XGBoost exposes dozens of knobs, AdaBoost stays elegantly simple — just four primary hyperparameters control its entire behaviour. Understanding the trade-off between learning speed and model complexity is the key to unlocking its real predictive power.

8.1 `base_estimator` — the foundational model

AdaBoost is an ensemble technique, so it needs a base algorithm to build on.

- **Default:** a `DecisionTreeClassifier` with `max_depth = 1` (a decision stump).
- **Alternatives:** Logistic Regression or SVM can theoretically be used.
- **Not allowed:** K-Nearest Neighbors (KNN) — it doesn't support sample weighting, which AdaBoost's reweighting mechanism absolutely depends on.

PRO TIP

In 99.9% of real-world applications, leave this as the default decision stump — the algorithm was designed and optimised around this specific weak learner.

8.2 `n_estimators` — model complexity vs. overfitting

The maximum number of stumps AdaBoost will sequentially train. If a perfect fit is reached earlier, training stops.

N_ESTIMATORS = 1 — UNDERFITTING

A single stump draws one rigid straight line. Textbook underfitting.

N_ESTIMATORS = 1500 — OVERFITTING

Hundreds of stumps bend the boundary into tiny isolated "islands" around single stray points — the model is memorising training data and will generalise poorly.

The goal: find the middle ground where the boundary is flexible enough to capture the real pattern, but smooth enough to generalise to unseen data.

8.3 `learning_rate` — the shrinkage parameter

The single most mathematically important lever for controlling overfitting caused by a high `n_estimators`.

$$\text{effective } \alpha = \text{learning_rate} \times \alpha \ , \quad \text{where } \alpha = \frac{1}{2} \ln \frac{1 - \text{error}}{\text{error}}$$

Default `learning_rate = 1.0`. Lower it to 0.1, and every stump's effective α shrinks by a factor of 10. Because α also drives the weight-update step (correct rows × $e^{-\alpha}$, wrong rows × e^{α}), a smaller α means gentler weight swings — misclassified points are penalised less severely, so each new boundary changes less drastically. This deliberate slowdown forces the ensemble toward smoother, more generalised boundaries — directly counteracting the overfitting risk of a high `n_estimators`.

THE CLASSIC TRADE-OFF

Low `learning_rate` + high `n_estimators` is the standard boosting recipe: many small, careful steps generalise better than few large, aggressive ones.

8.4 `algorithm` — SAMME vs. SAMME.R

Option	Meaning	Notes
SAMME.R	"Real" — uses class *probabilities* instead of hard predictions	Default & recommended; converges faster, typically lower test error
SAMME	Discrete version — uses hard class predictions	Compare both via cross-validation if unsure

PART 09 · ADVANCED

GridSearchCV in Practice

To find the sweet spot between `n_estimators` and `learning_rate`, sweep them systematically with `GridSearchCV`.

9.1 Baseline

Running AdaBoost with pure defaults —

`max_depth = 1` `n_estimators = 50` `learning_rate = 1.0` `algorithm = SAMME.R`

— might yield a baseline accuracy around **78%**.

9.2 Build the grid

```
param_grid = {
    'n_estimators':   [50, 100, 500, 1500],
    'learning_rate':  [0.001, 0.01, 0.1, 1.0],
    'algorithm':      ['SAMME', 'SAMME.R']
}

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import AdaBoostClassifier

grid = GridSearchCV(
    AdaBoostClassifier(),
    param_grid,
    cv=10          # 10-fold cross-validation
)
grid.fit(X_train, y_train)
```

9.3 The result

Pushing this grid through 10-fold cross-validation systematically tests every combination. In this scenario, pairing a **high estimator count (500)** with a **low learning rate (0.1)** under SAMME.R smoothed the decision boundary and lifted accuracy:

	n_estimators	learning_rate	algorithm	Accuracy
Baseline (defaults)	50	1.0	SAMME.R	78%
Best found	**500**	**0.1**	**SAMME.R**	**83%**

TAKEAWAY

A five-point accuracy jump came purely from balancing more (but smaller) sequential corrections against each other — the shrinkage/estimator-count trade-off from Part 8 in action.

PART 10 · ADVANCED / PRO

Bagging vs. Boosting — The Real Difference

Every ensemble method chases the same goal: low **bias** (few mistakes on training data) and low **variance** (consistent performance on unseen data). Bagging (e.g. Random Forest) and Boosting (e.g. AdaBoost) reach that goal from completely opposite directions.

10.1 Type of base model (the bias-variance trade-off)

	Bagging	Boosting
Base model	Fully grown, deep decision trees	Shallow decision stumps (depth = 1)
Individual bias	**Low — near-zero training error**	**High — makes many mistakes**
Individual variance	**High — fluctuates wildly on new data**	**Low — predictable across datasets**
Fix applied	Averages many high-variance trees together	Combines hundreds of stumps that correct each other's mistakes

10.2 Parallel vs. sequential learning

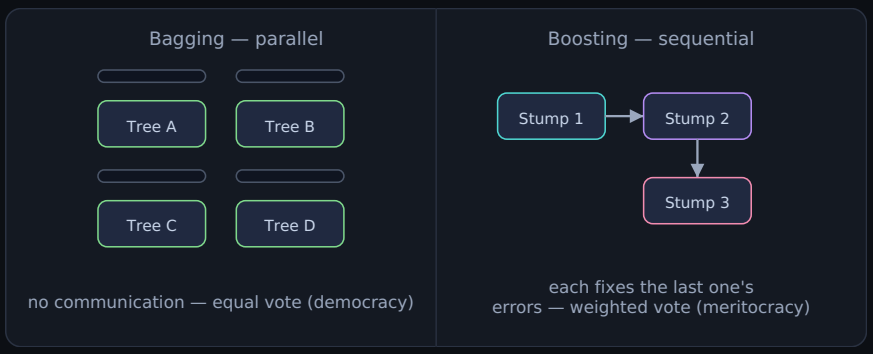


Fig 10.1 — Bagging trains independent trees on random sub-samples simultaneously with no communication. Boosting trains one stump at a time, each conditioned on the last one's mistakes.

- **Bagging (Bootstrap Aggregation):** completely parallel. The dataset is randomly sub-sampled, and those portions are distributed to hundreds of independent trees that train simultaneously.
- **Boosting:** strictly sequential. Data flows to Model 1 → its mistakes reshape the dataset → Model 2 trains on the reshaped data and tries to fix those mistakes → the chain continues.

10.3 Voting structure — democracy vs. meritocracy

BAGGING IS A DEMOCRACY

Every fully grown tree gets exactly one equal vote. If 300 trees vote "Yes" and 200 vote "No", the answer is "Yes" — regardless of any individual tree's accuracy.

BOOSTING IS A MERITOCRACY

Base models are assigned distinct weights (α) based on individual accuracy. A highly accurate stump gets a massive mathematical say; a poor stump's vote is heavily discounted — exactly what you saw in Part 6.

PART 11 · REFERENCE

Cheat Sheet & Where to Go Next

11.1 The whole algorithm in one loop

#	Action
1	Initialise every row's weight to 1/N
2	Train a decision stump on the (weighted) data
3	Compute weighted error = sum of weights of misclassified rows
4	Compute α = ½ ln((1−error)/error)
5	Update row weights: correct × $e^{-\alpha}$, wrong × e^{α}
6	Normalise weights to sum to 1
7	Upsample using the weights to build the next stage's dataset
8	Repeat steps 2–7 for T stumps (n_estimators)
9	Predict with H(x) = sign(Σ α_t $h_t(x)$)

11.2 Key formulas, together

$$\text{error} = \Sigma \text{ (weights of misclassified rows)}$$

$$\alpha = \frac{1}{2} \ln \frac{1 - \text{error}}{\text{error}}$$

$$w_{new} = w_{old} \times e^{-\alpha} \quad \text{(correct)} \qquad | \qquad w_{new} = w_{old} \times e^{+\alpha} \quad \text{(wrong)}$$

$$H(x) = \text{sign} \left(\Sigma_{t=1}^{T} \alpha_t \, h_t(x) \right)$$

11.3 Where to go from here

NEXT LOGICAL STEP

You now understand how stage-wise boosting identifies mistakes and mathematically corrects for them. The natural next stop is **Gradient Boosting**: instead of reweighting misclassified rows, it fits each new model directly to the *residual errors* of the previous one — a generalisation of exactly the correction loop you just walked through by hand.

If you want to understand…	Study next
Regression-style boosting via residuals	Gradient Boosting Machines (GBM)
Regularised, production-grade boosting	XGBoost / LightGBM
Why deep trees + averaging works so well	Random Forest (revisit bagging in depth)

YOU MADE IT

From a single decision stump to a full weighted ensemble, a hand-computed worked example, and production-level hyperparameter tuning — that's the complete arc of AdaBoost, basic to pro.